

713. Union of Two Sorted Arrays (Two Pointers)

Platform: DSA Practice

EASY

10 Oct 2026

Problem Summary

Given two sorted arrays **nums1** and **nums2**, return an array of all **distinct** values present in either array, in **ascending** order.

Example: [3,4,4,4] and [6,7,7] gives [3,4,6,7].

Approach

Both arrays are already sorted, so merge them like merge sort with one pointer per array. Always take the smaller current element. Before adding it to the result, skip it if it equals the last value added. That removes repeats inside one array and across both.

Key Insights

- Sorted input means duplicates are adjacent, so checking only the last added value is enough.
- Main loop must use **&&**; **||** goes out of bounds once one array ends.
- Merging the equal case into **<=** removes the separate **n1++**, **n2++** branch.

Observations

- Works with negative values; only **<=** and **!=** are used.
- Integer vs int compare unboxes, so **!=** compares values here.
- Uses the sortedness hint, which interviewers expect.

Approaches

- 1. TreeSet** (simple): add every element to a TreeSet, which dedupes and sorts, then copy out.
- 2. Two Pointers** (optimal): merge the two sorted arrays, skipping repeats. <- [this page](#)

Algo

- Make list **lt**, pointers **n1 = 0**, **n2 = 0**.
- While **n1** and **n2** are both in range, compare **nums1[n1]** and **nums2[n2]**.
- Take the smaller one (on equal, take from **nums1**).
- Add it only if it is empty or **last(lt)** differs.
- Move the pointer you took from.
- Repeat the same add-with-check for leftovers of **nums1**, then **nums2**.
- Copy **lt** into an **int[]** and return it.

Dry Run

Step	Action	State
1	3 <= 6, lt empty, add 3	[3]
2	4 <= 6, add 4	[3,4]
3	4 <= 6, last is 4, skip	[3,4]
4	4 <= 6, last is 4, skip	[3,4]
5	nums1 done; leftovers of nums2: add 6, add 7, skip 7	[3,4,6,7]

Time & Space Complexity

Time: $O(n1 + n2)$

Space: $O(n1 + n2)$ for the result list (the output itself has that size).

Points to Remember

- Two pointers on sorted arrays gives a merge-style $O(n1 + n2)$ solution.
- Dedupe sorted data by comparing with the last kept element.
- Main loop with **&&**, then separate leftover loops.
- List<Integer>** to **int[]** needs a manual loop or **mapToInt**.

Solution Code

```
class Solution {
    public int[] unionArray(int[] nums1, int[] nums2) {
        List<Integer> lt = new ArrayList<>();
        int n1 = 0, n2 = 0;
        while (n1 < nums1.length && n2 < nums2.length) {
            int x = (nums1[n1] <= nums2[n2]) ? nums1[n1++] : nums2[n2++];
            if (lt.isEmpty() || lt.get(lt.size() - 1) != x) lt.add(x);
        }
        while (n1 < nums1.length) {
            int x = nums1[n1++];
            if (lt.isEmpty() || lt.get(lt.size() - 1) != x) lt.add(x);
        }
        while (n2 < nums2.length) {
            int x = nums2[n2++];
            if (lt.isEmpty() || lt.get(lt.size() - 1) != x) lt.add(x);
        }
        int[] arr = new int[lt.size()];
        for (int i = 0; i < lt.size(); i++) arr[i] = lt.get(i);
        return arr;
    }
}
```

713. Union of Two Sorted Arrays (TreeSet)

Platform: DSA Practice

EASY

10 Oct 2026

Problem Summary

Given two sorted arrays **nums1** and **nums2**, return an array of all **distinct** values present in either array, in **ascending** order.

Example: [3,4,4,4] and [6,7,7] gives [3,4,6,7].

Approach

Insert every element of both arrays into a **TreeSet**. A set drops duplicates and a **TreeSet** keeps its elements sorted, so iterating it gives the union directly. Copy the values into an `int[]`.

Key Insights

- **TreeSet** gives uniqueness and sorted order in one structure.
- It ignores the fact that the inputs are already sorted.
- **HashSet** would dedupe but would not keep order.

Observations

- Shorter and quicker to write than two pointers.
- Also works when the arrays are unsorted.
- Fine for n up to 1000, slower than two pointers in theory.

Approaches

- **1. TreeSet** (simple): add every element to a **TreeSet**, which dedupes and sorts, then copy out. <- **this page**
- **2. Two Pointers** (optimal): merge the two sorted arrays, skipping repeats.

Algo

1. Create a **TreeSet**<Integer>.
2. Add every element of `nums1`.
3. Add every element of `nums2`.
4. Create `int[]` arr of size `set.size()`.
5. Iterate the set and fill arr in order.
6. Return arr.

Dry Run

Step	Action	State
1	Add <code>nums1</code> : 3, 4, 4, 4	{3,4}
2	Add <code>nums2</code> : 6, 7, 7	{3,4,6,7}
3	Copy set into arr in order	[3,4,6,7]

Time & Space Complexity

Time: $O((n1 + n2) \log(n1 + n2))$, since each **TreeSet** insert is $O(\log n)$.

Space: $O(n1 + n2)$ for the set, plus the output array.

Points to Remember

- **TreeSet** = dedupe + sort, at $O(\log n)$ per insert.
- **HashSet** has no ordering guarantee, so it needs a sort afterwards.
- Good quick answer for unsorted input; for sorted input prefer two pointers.
- Be ready to explain why two pointers beats it in an interview.

Solution Code

```
class Solution {
    public int[] unionArray(int[] nums1, int[] nums2) {
        TreeSet<Integer> set = new TreeSet<>();

        for (int n : nums1) {
            set.add(n);
        }
        for (int n : nums2) {
            set.add(n);
        }

        int[] arr = new int[set.size()];
        int i = 0;
        for (int n : set) {
            arr[i] = n;
            i++;
        }
        return arr;
    }
}
```